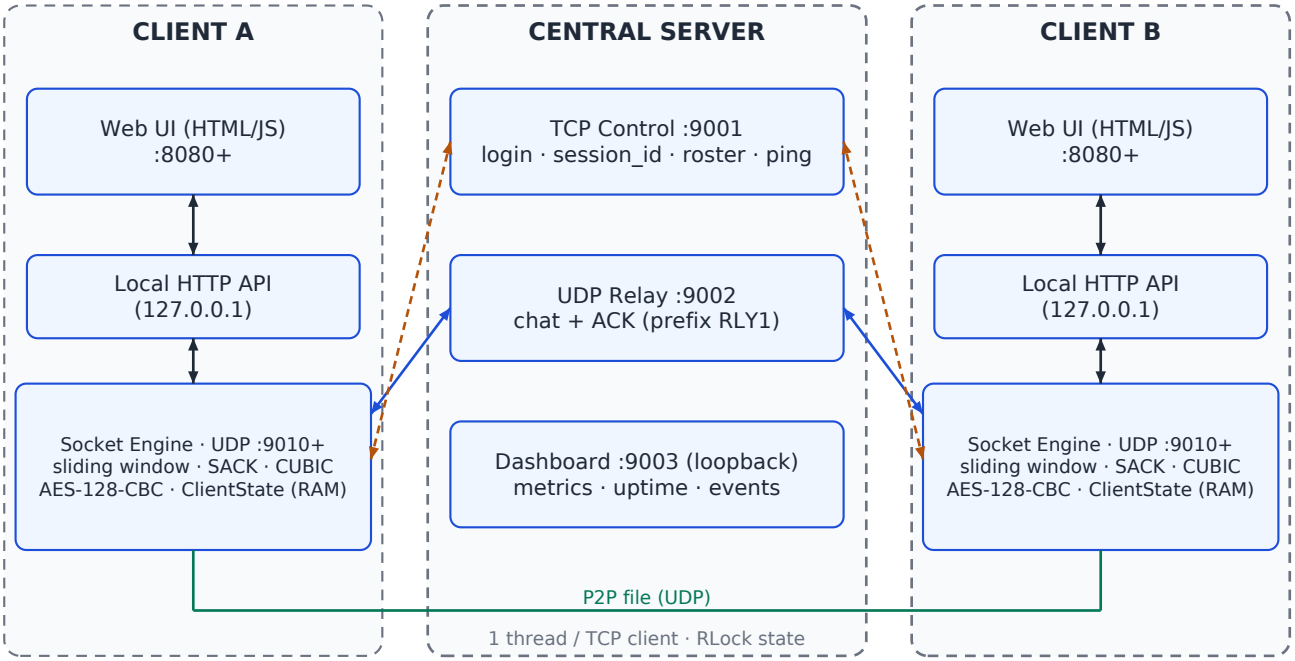
Technical Explanation Report

Distributed and Streaming-based Client-Server Collaboration Application

Môn: Network Application Development (CO3093/CO3094) · HCMUT
Sinh viên: **[Họ tên – MSSV]** · Ngôn ngữ: Python 3 (socket thô, không dùng framework mạng cấp cao ngoài `http.server` cho giao diện cục bộ)

1. System Architecture

Hệ thống kiến trúc **lai (hybrid)**: TCP chỉ dùng cho mặt phẳng điều khiển (đăng nhập, cấp session_id, danh bạ online, phát hiện client chết); UDP dùng cho mặt phẳng dữ liệu. Chat đi qua **UDP relay** của server; file đi **P2P trực tiếp** giữa hai client để server không là nút nghẽn băng thông. Ba lớp được tách rõ: **mạng** (`network/`), **xử lý dữ liệu** (`core/`: packet, crypto, state) và **giao diện** (`web/`).



Thành phần	Giao thức / cổng	Vai trò
Server TCP Control	TCP 9001	Login, cấp session_id, roster/presence, nhận ping; mỗi client một thread, timeout 15 s
Server UDP Relay	UDP 9002	Xác minh (session, IP, port) rồi chuyển tiếp gói chat/ACK trong phong bì RLY1
Dashboard	HTTP 127.0.0.1:9003	Metrics, uptime, tối đa 100 sự kiện (không ghi nội dung chat)
Client Socket Engine	UDP 9010+	Đóng/mở gói, cửa sổ trượt, SACK, CUBIC, mã hóa, nhận file P2P
Client Web UI	HTTP 127.0.0.1:8080+	HTML/JS thuần: chat, danh sách online, thanh tiến độ file

2. Protocol Specification

Mọi gói tin bắt đầu bằng header nhị phân cố định **24 byte**, big-endian (`struct '!HBBIIIIHH'`), theo sau là payload.

Trường	Byte	Kiểu	Mô tả
Magic	0–1	uint16	0x4E41 ("NA") – lọc gói rác
Version	2	uint8	0x01
Flags	3	uint8	Bitmask loại gói (bảng dưới)
Session ID	4–7	uint32	Do server cấp khi login
Stream ID	8–11	uint32	Kênh logic: 0 = chat chung; chat riêng = CRC-32("nhỏ:lớn"); file = ngẫu nhiên ≥ 2
Sequence	12–15	uint32	Tăng dần theo (peer, stream), bắt đầu từ 1; ACK có seq = 0
ACK No	16–19	uint32	Seq kế tiếp bên nhận mong đợi (cumulative ACK)
Payload Len	20–21	uint16	Độ dài payload
Checksum	22–23	uint16	16 bit thấp CRC-32 trên header (checksum = 0) + payload

Cờ điều khiển (Flags)

Cờ	Hex	Ý nghĩa	Cờ	Hex	Ý nghĩa
AUTH	0x01	dự phòng (chưa dùng)	ACK	0x10	Báo nhận + SACK
DISCOV	0x02	Discovery (P2P cũ)	HEARTBEAT	0x20	Giữ kết nối (P2P cũ)
MSG	0x04	Tin nhắn chat	SYN	0x40	dự phòng (chưa dùng)
FILE	0x08	Metadata / chunk file	FIN	0x80	dự phòng (chưa dùng)

Payload

- **MSG / FILE:** mã hóa AES-128-CBC, PKCS#7, IV ngẫu nhiên 16 B đặt đầu: `[IV 16B][ciphertext]`. Header và ACK **không** mã hóa.
- **FILE:** gói đầu là JSON metadata `{filename, filesize, total_chunks, sha256, transfer_id, chat_stream_id}`; các gói sau là chunk ≤ 1350 B (sau mã hóa ≤ 1376 B → datagram ≈ 1428 B < MTU 1500).
- **ACK:** `[ver uint8][n uint8]` + tối đa 32 cặp `[left, right)` uint32 – các đoạn đã nhận nằm sau lỗ hổng đầu tiên (SACK).

Phong bì UDP relay (RLY1)

`"RLY1" + JSON {type: relay|deliver, sender_session_id, target_session_id, packet: base64(gói giao thức)}`. `target_session_id = 0` là broadcast. Server kiểm tra session, IP và UDP port nguồn trước khi chuyển tiếp; gói file không qua relay.

Bản tin TCP (JSON, kết thúc bằng newline)

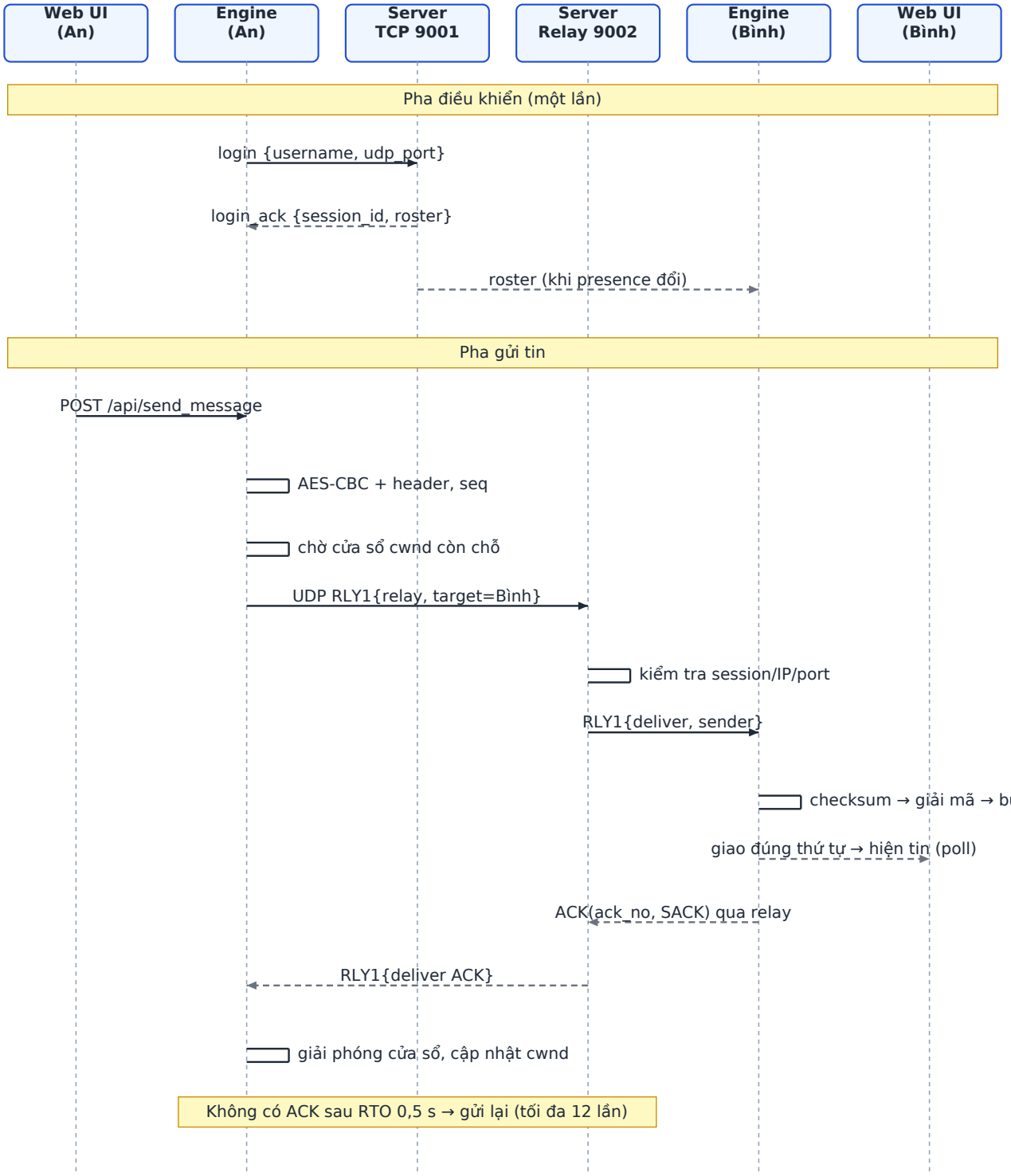
Hướng	Bản tin
Client → Server	`{"type":"login","username":"An","udp_port":9010}`; `{"type":"ping"}` mỗi 3 s
Server → Client	`login_ack {success, session_id, users}`; `roster {users:[…]}` khi presence đổi

Độ tin cậy và điều khiển tắc nghẽn

- **Sliding window:** cửa sổ tối đa 8 gói; bên nhận (StreamReceiver) giữ buffer ngoài thứ tự và chỉ giao dữ liệu liên tục theo từng stream.
- **Retransmission:** RTO 0,5 s; fast retransmit khi có ≥ 2 gói phía sau đã được SACK; chỉ gửi lại gói thiếu; bỏ cuộc sau 12 lần (MAX_RETRIES).
- **CUBIC:** cwnd khởi tạo 4, tối thiểu 2, β = 0,7, C = 0,4; $W_{cubic}(t) = C(t - K)^3 + W_{max}$ với $K = \sqrt[3]{W_{max}(1 - \beta)/C}$; có vùng TCP-friendly; giảm cwnd một lần mỗi đợt mất gói.

3. Sequence Diagram – vòng đời một tin nhắn

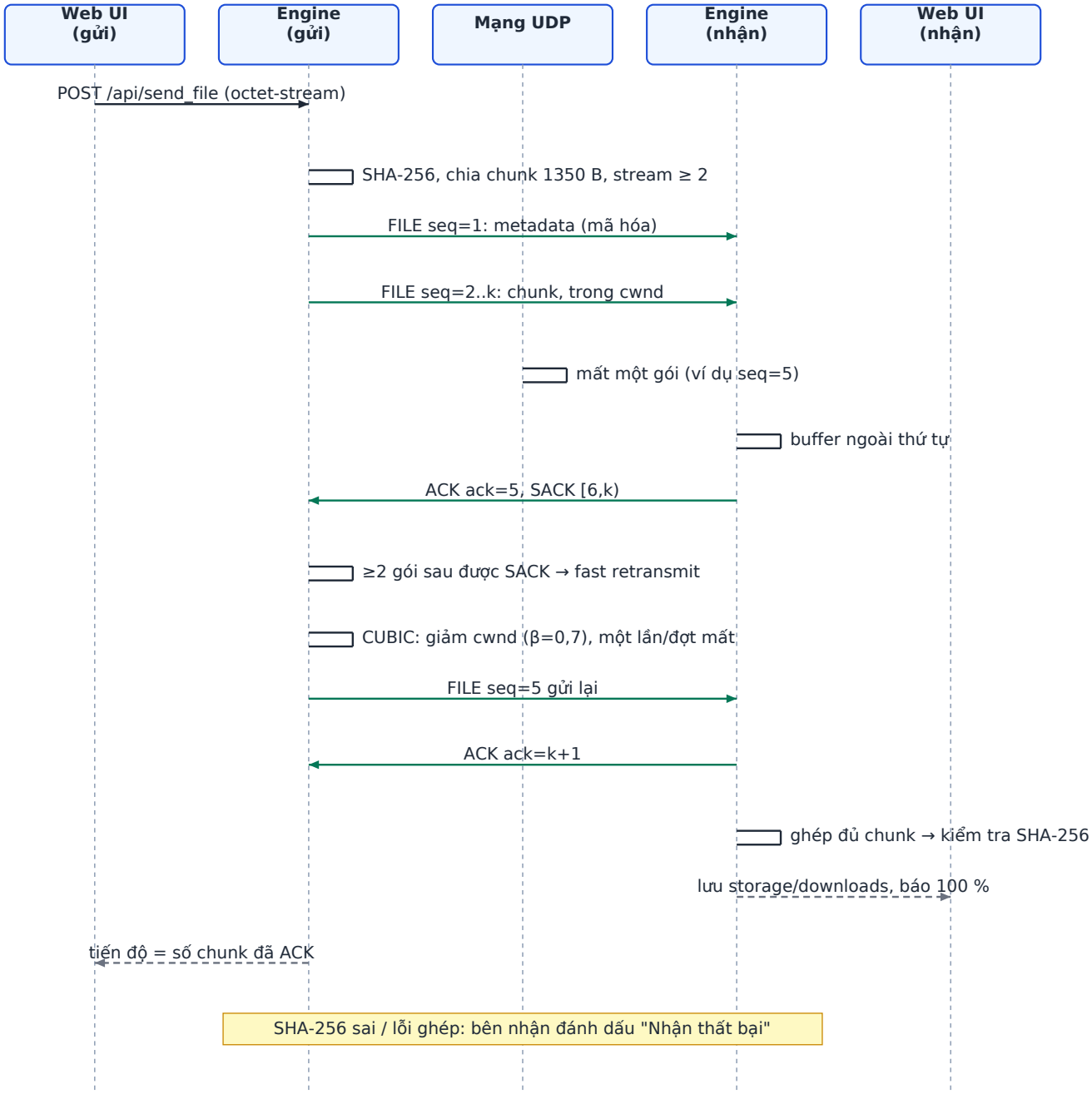
Ví dụ An gửi chat riêng cho Bình qua UDP relay. Tin nhắn chỉ xuất hiện trong lịch sử của Bình khi đã được giao **đúng thứ tự** trong stream của nó.



Gói chat chung (stream 0) đi cùng đường nhưng `target_session_id = 0`; server nhân bản tới mọi client online. Nếu một client im lặng quá 15 s, server đóng TCP, loại khỏi roster và gửi roster mới cho các client còn lại.

4. Sequence Diagram – vòng đời một lần truyền file

File đi **P2P trực tiếp** (không qua server) trên một stream riêng, nên chat vẫn chạy bình thường trong lúc file đang chờ gửi lại gói mất (chống Head-of-Line blocking).



Nếu bên nhận biến mất, một gói bị gửi lại quá 12 lần → bộ gửi đánh dấu failed, `send_file` báo `ConnectionError` và giao diện hiển thị lỗi thay vì treo vô hạn.

5. Challenges & Solutions

Thách thức	Giải pháp
Head-of-Line blocking: file lớn mất gói làm chat bị chặn nếu dùng một chuỗi seq chung.	Mỗi cuộc hội thoại/file là một stream riêng với seq, bộ nhận và cửa sổ riêng. Test e2e: 5 tin chat đến trong 0,5 s khi file đang ở trạng thái RECEIVING.
Mất gói UDP: UDP không đảm bảo giao hàng, nhưng chat và file cần tin cậy.	Seq + cumulative ACK + SACK (≤ 32 block); chỉ gửi lại gói thiếu; RTO 0,5 s và fast retransmit sau 2 SACK. File 1 MB vẫn đúng SHA-256 với 10 % mất gói giả lập.
Tắc nghẽn: gửi tối đa tốc độ làm tràn đường truyền và gây mất gói thêm.	Cửa sổ trượt giới hạn 8 gói và CUBIC (RFC 8312) kèm vùng TCP-friendly; chỉ giảm cwnd một lần cho mỗi đợt mất gói để khỏi sụp cửa sổ.
Phân mảnh IP: datagram lớn hơn MTU bị phân mảnh, mất một mảnh là mất cả gói.	Chunk file 1350 B; sau AES-CBC/PKCS#7 ≤ 1376 B; cộng header 24 B + UDP/IP = 1428 B < 1500 B.
Race condition: luồng nhận UDP, luồng gửi lại, luồng TCP và HTTP server cùng sửa trạng thái.	ClientState bảo vệ bằng RLock; mỗi client TCP một thread; vòng gửi lại chạy thread riêng và gỡ các bộ gửi lỗi một cách an toàn.
Client chết đột ngột (rút mạng, kill tiến trình): TCP không báo ngay, roster sai.	Client gửi ping mỗi 3 s; server đặt timeout 15 s, đóng kết nối, loại user và phát roster mới. Có test với `client_timeout = 1 s`.
Bên nhận biến mất giữa chừng: bộ gửi gửi lại mãi, giao diện treo.	Giới hạn `MAX_RETRIES = 12`; vượt giới hạn thì hủy stream, báo lỗi cho UI.
Gói rác / hỏng / giải mã lỗi làm sập luồng nhận.	Kiểm tra Magic + checksum trước, bọc giải mã và xử lý chunk trong try/except; chunk lỗi đánh dấu transfer ERROR, server không crash.
Va chạm Stream ID giữa chat riêng (băm CRC-32) và file (ngẫu nhiên).	Chấp nhận rủi ro rất nhỏ (không gian 2^{32}), ghi rõ trong tài liệu; hướng cải tiến: server cấp stream ID.
Input không tin cậy ở HTTP API (JSON hỏng, body quá lớn).	Giới hạn 1 MB (JSON) / 200 MB (upload), trả 400/413/500 có kiểm soát; HTTP chỉ bind 127.0.0.1.

6. Kiểm thử và hạn chế

Kết quả kiểm thử

- `python -m unittest discover -s test -t . -v` → **37 test** (cửa sổ trượt, CUBIC, SACK, giới hạn retry, bắt tay TCP, timeout client, AES, e2e có mất gói).
- `python tools/load_test.py --clients 20 --file-mb 1` → 20 client đồng thời, file 1 MB: PASS.

[Dán ảnh chụp log test, load test và giao diện Web UI (danh sách online, chat, thanh tiến độ file) vào đây.]

Hạn chế đã biết

- Khóa AES dùng chung (biến môi trường `CNET_SECRET_KEY`), không trao đổi khóa; CBC không có MAC nên không chống sửa đổi chủ động.
- Header, ACK và gói điều khiển không được mã hóa.
- Bên gửi không được báo khi bên nhận lỗi ghép file; sau khi hủy, bộ gửi mới bắt đầu lại từ seq 1.
- RTT trong CUBIC được ước lượng từ RTO (không đo thực tế); `to_json_snapshot` chỉ là bản sao nông.
- Giao diện dùng `http.server` của thư viện chuẩn (không framework); không có xác thực/CSRF vì chỉ bind loopback.